EXAM-ORIENTED STUDY GUIDE

C Fundamentals

Character Set · Data Types · Operators · Preprocessor · Program Translation

A concise yet complete revision resource covering the C character set, identifiers & keywords, data types & constants, type qualifiers, escape sequences, arithmetic expressions & operators, preprocessor directives, header files, symbolic constants, comments, the `sizeof` operator, and the complete translation process of a C program — with labelled diagrams, comparison tables, worked examples and quick-revision notes for university and competitive examinations.

Table of Contents

Chapter 1 Introduction to C Language

Chapter 2 C Character Set

Chapter 3 Identifiers and Keywords

Chapter 4 Data Types and Constants

Chapter 5 Type Qualifiers

Chapter 6 Escape Sequences

Chapter 7 Arithmetic Expressions and Operators

Chapter 8 Preprocessor Directives

Chapter 9 Header Files

Chapter 10 Symbolic Constants

Chapter 11 Comments in C

Chapter 12 The sizeof Operator

Chapter 13 Translation Process of a C Program

Chapter 14 Quick Revision Section

1 Introduction to C Language

1.1 History of C

C was developed by **Dennis Ritchie** at **Bell Laboratories (AT&T)** in the year **1972**. It evolved from two earlier languages — **BCPL** (developed by Martin Richards) and **B** (developed by Ken Thompson). C was originally created to re-write the UNIX operating system. In **1988**, the American National Standards Institute (ANSI) standardized the language, giving rise to **ANSI C (C89/C90)**.

1.2 Features of C

- **Middle-level language** — combines features of both low-level and high-level languages.
- **Structured language** — programs are divided into functions/blocks.
- **Portable** — C programs can run on different machines with little or no modification.
- **Fast and efficient** — produces compact, quick executable code.
- **Rich set of operators** and built-in functions.
- **Extensible** using functions and libraries.
- Supports **pointers** for direct memory access.
- **Case-sensitive** and uses a free-form line structure.

1.3 ANSI C Standard

ANSI C (also called C89/C90) is the standardized version of C ratified by the American National Standards Institute in 1989 (adopted by ISO in 1990). It standardized the syntax, keywords, library functions, and data types so that C programs would be portable across compilers and platforms.

> **Exam Tip**
>
> ANSI C defines exactly **32 keywords**. Later standards (C99, C11) added more keywords, but syllabus-level questions on "Keywords under ANSI C" expect the original 32.

1.4 Applications of C Programming

Domain	Examples / Use
Operating Systems	UNIX, Linux kernel, parts of Windows
System Software	Compilers, interpreters, device drivers
Embedded Systems	Microcontroller firmware, IoT devices
Application Software	Database systems (MySQL core), utilities
Language Development	Base for C++, Java, Python (CPython), C#

1.5 Basic Structure of a C Program

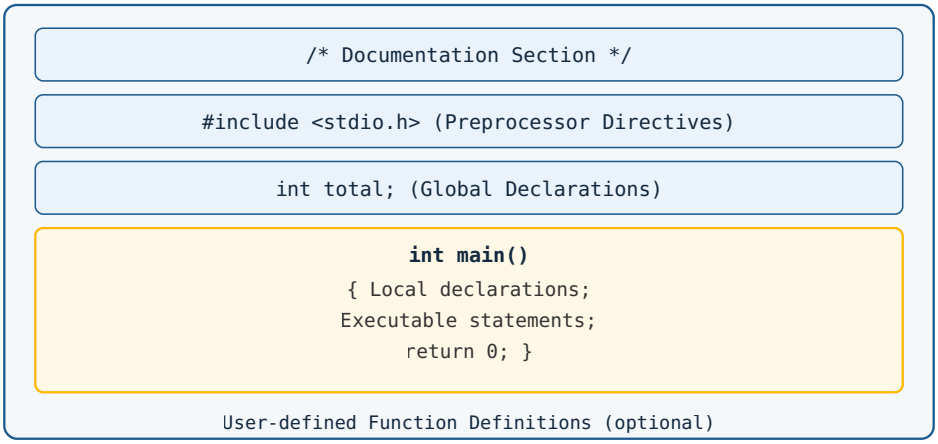


Fig. 1.1 — Basic structure / layout of a C program

Section	Purpose
Documentation	Comments describing program purpose, author, date (ignored by compiler)
Preprocessor Directives	`#include`, `#define` — processed before compilation
Global Declarations	Variables/functions accessible throughout the file
`main()` function	Mandatory entry point; execution always begins here
User-defined functions	Additional functions called from `main()` or elsewhere

Chapter Summary

- C was created by Dennis Ritchie (1972) at Bell Labs, evolved from BCPL → B → C.
- ANSI standardized C in 1989 (ANSI C / C89 / C90) with 32 keywords.
- C is a middle-level, structured, portable, and efficient programming language.
- Widely used in OS, embedded systems, compilers, and as the base for many modern languages.
- Every C program follows: Documentation → Preprocessor directives → Global declarations → `main()` → User-defined functions.

2 C Character Set

The **character set** of C is the set of valid characters that a C compiler can recognize. A character represents any letter, digit, or symbol used to represent information. C source programs are written using these characters.

2.1 Components of the C Character Set

(a) Alphabets

Uppercase `A–Z` and lowercase `a–z` letters (52 letters total). C is case-sensitive, so `Sum` and `sum` are different identifiers.

(b) Digits

Decimal digits `0–9` (10 digits), used to form numeric constants and as part of identifiers (not as the first character).

(c) Special Symbols

Symbol	Name	Symbol	Name
,	Comma	&	Ampersand
.	Period / dot	^	Caret
;	Semicolon	*	Asterisk
:	Colon	-	Minus sign
?	Question mark	+	Plus sign
'	Apostrophe	< >	Angle brackets
"	Quotation mark	( )	Parentheses
!	Exclamation mark	[ ]	Square brackets
|	Vertical bar	{ }	Curly braces
/	Slash	%	Percent sign
\	Backslash	#	Hash / pre-processor
~	Tilde	_ $ =	Underscore, dollar, equal-to

(d) White Spaces

White space includes the **blank space**, **horizontal tab**, **new line**, and **carriage return**. White spaces are used to separate tokens (keywords, identifiers, constants) in a program and are ignored by the compiler outside of string/character constants — they exist purely to improve readability.

2.2 Usage and Importance

- Every C program, statement, token, and identifier is built entirely from this character set.
- Special symbols form **operators** and **punctuation** (e.g. `;` ends a statement, `{ }` mark a block).
- Any character outside this set (used incorrectly) causes a **compilation error**.

> **Note**
>
> The C character set is distinct from the extended ASCII set that may appear inside string/character constants (e.g. inside a string literal, virtually any character can appear).

Chapter Summary

- Character set = Alphabets (A-Z, a-z) + Digits (0-9) + Special Symbols + White Spaces.
- C is case-sensitive.
- White spaces separate tokens and are ignored by the compiler (except inside strings).

3 Identifiers and Keywords

3.1 Identifiers

Definition: An identifier is a name given to program elements such as variables, functions, arrays, structures, and labels, so that they can be referred to elsewhere in the program.

Rules for Naming Identifiers

1. May consist only of alphabets, digits, and the underscore `_`.
2. The first character must be a letter or an underscore — it **cannot** be a digit.
3. No special symbols or white space are allowed (except the underscore).
4. Keywords cannot be used as identifiers.
5. C is case-sensitive, so `Total` and `total` are different identifiers.
6. ANSI C guarantees only the first **31 characters** of an identifier are significant.

Valid and Invalid Examples

Identifier	Valid?	Reason
`total_marks`	Valid	Letters and underscore only
`_temp`	Valid	Starts with underscore
`sum1`	Valid	Digit not in first position
`1sum`	Invalid	Starts with a digit
`int`	Invalid	Reserved keyword
`roll no`	Invalid	Contains a space
`rate%`	Invalid	Contains special symbol `%`

3.2 Keywords

Definition: Keywords are reserved words that have a pre-defined, fixed meaning in the C language. They cannot be used as variable names, function names, or any other identifier.

> **Note**
>
> ANSI C defines exactly **32 keywords**, all in lowercase.

The 32 ANSI C Keywords

`auto`	`break`	`case`	`char`
`const`	`continue`	`default`	`do`
`double`	`else`	`enum`	`extern`
`float`	`for`	`goto`	`if`
`int`	`long`	`register`	`return`
`short`	`signed`	`sizeof`	`static`
`struct`	`switch`	`typedef`	`union`
`unsigned`	`void`	`volatile`	`while`

3.3 Comparison Table: Identifiers vs Keywords

Basis	Identifier	Keyword
Definition	User-defined name for a program element	Reserved word with fixed compiler meaning
Who defines it	Programmer	C Language itself
Count	Unlimited (user chosen)	Fixed — 32 in ANSI C
Case	Any combination of case	Always lowercase
Reusability as a name	Can be reused/redeclared in different scopes	Cannot be used as a variable/function name
Example	`marks`, `_total`	`int`, `while`, `return`

Common Mistake

Using a keyword as a variable name (e.g. `int float = 5;`) — this always produces a compilation error.

Chapter Summary

- Identifiers = user-defined names; must start with a letter/underscore, contain only letters, digits, underscore.
- Keywords = 32 reserved, fixed-meaning, lowercase words in ANSI C — cannot be redefined.
- Only the first 31 characters of an identifier are guaranteed significant under ANSI C.

4 Data Types and Constants

4.1 What is a Data Type?

A **data type** specifies the type of data a variable can hold, how much memory it occupies, and the range of values and operations permitted on it. C provides basic (primary), derived, and user-defined data types.

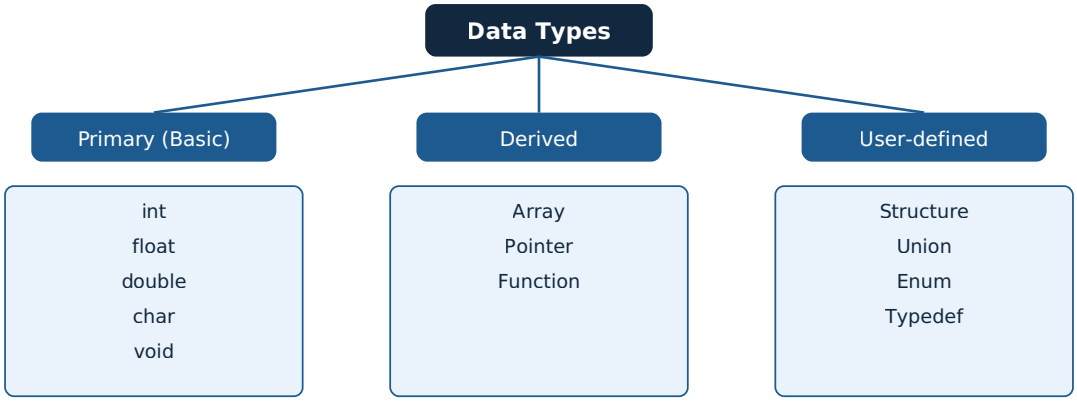


Fig. 4.1 — Classification of Data Types in C

4.2 Basic (Primary) Data Types

Data Type	Description	Typical Size	Format Specifier
`int`	Whole numbers (integers)	2 or 4 bytes	`%d`
`float`	Single-precision floating point (decimal) numbers	4 bytes	`%f`
`double`	Double-precision floating point numbers (higher accuracy)	8 bytes	`%lf`
`char`	Single character	1 byte	`%c`
`void`	Represents "no value" — used for functions that return nothing, or generic pointers	—	—

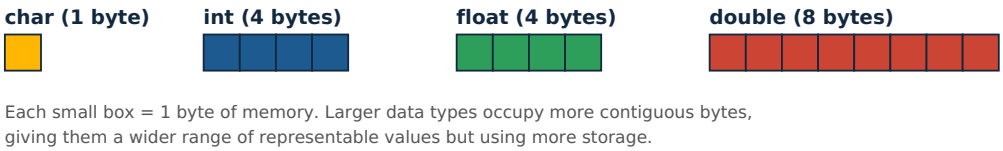
> **Note**
> Exact sizes are compiler/platform dependent; the ANSI C standard only guarantees minimum ranges, not exact byte counts. Sizes above are the common values on 32/64-bit systems.

Brief Overview — Derived and User-defined Types

- **Derived types** are built from basic types: `array` (collection of same-type elements), `pointer` (stores an address), `function`.

- **User-defined types** let the programmer create custom types: `struct` (group of different data types), `union` (shares memory among members), `enum` (named integer constants), `typedef` (creates an alias for an existing type).

4.3 Memory Representation of Data Types



Each small box = 1 byte of memory. Larger data types occupy more contiguous bytes, giving them a wider range of representable values but using more storage.

Fig. 4.2 — Relative memory occupied by basic data types

4.4 Constants

A **constant** is a value that does not change during program execution.

Type of Constant	Description	Examples
Integer Constant	A whole number without decimal point	`10`, `-45`, `0`
Floating-point Constant	A number with a decimal point or exponent (used for `float` / `double`)	`3.14`, `2.5e3`
Character Constant	A single character enclosed in single quotes	`'A'`, `'5'`, `'\n'`
String Constant	Sequence of characters enclosed in double quotes	`"Hello"`

4.5 Variables and Initialization

SYNTAX

```
data_type variable_name = value;
```

EXAMPLE

```
int   age   = 21;
float price = 99.5;
char  grade = 'A';
double pi   = 3.14159;
```

A variable must be **declared** before use, stating its data type. Initialization assigns an initial value at the time of declaration; if omitted, the variable holds an indeterminate (garbage) value until assigned.

Chapter Summary

- Data types = Primary (int, float, double, char, void), Derived (array, pointer, function), User-defined (struct, union, enum, typedef).
- Constants: integer, floating-point, character, and string — values that never change.
- Variables must be declared with a data type before use; can be initialized at declaration.

5 Type Qualifiers

Definition: Type qualifiers (also called **modifiers**) are keywords used along with basic data types to alter the size, range, or sign-handling of the data that a variable can store. C provides four type qualifiers: `short`, `long`, `signed`, and `unsigned`.

5.1 short

Reduces the storage size (and hence range) of an integer type. `short int` typically uses **2 bytes**.

5.2 long

Increases the storage size (and hence range) of an integer or double type. `long int` typically uses **4 bytes**; `long double` typically uses **8–16 bytes** (compiler dependent).

5.3 signed

Allows a variable to store both **positive and negative** values. This is the **default** for integer and character types if no qualifier is specified.

5.4 unsigned

Restricts a variable to store **only non-negative (zero and positive)** values, effectively doubling the maximum positive range compared to the signed version of the same size.

SYNTAX

```
[signed|unsigned] [short|long] data_type variable_name; Examples: unsigned int marks; short int count; long int population; unsigned char flag;
```

5.5 Range and Storage Comparison Table

Data Type	Typical Size	Range (typical, 32/64-bit systems)
`char` / `signed char`	1 byte	-128 to 127
`unsigned char`	1 byte	0 to 255
`short int`	2 bytes	-32,768 to 32,767
`unsigned short int`	2 bytes	0 to 65,535
`int`	4 bytes	-2,147,483,648 to 2,147,483,647
`unsigned int`	4 bytes	0 to 4,294,967,295
`long int`	4 bytes	-2,147,483,648 to 2,147,483,647
`unsigned long int`	4 bytes	0 to 4,294,967,295
`float`	4 bytes	~3.4E-38 to 3.4E+38 (6 digit precision)
`double`	8 bytes	~1.7E-308 to 1.7E+308 (15 digit precision)
`long double`	8–16 bytes	~3.4E-4932 to 1.1E+4932 (compiler dependent)

Exam Tip

Exact sizes/ranges are implementation (compiler) dependent per the ANSI C standard — always use the `sizeof` operator to verify on a given system. Values above reflect the common convention on most 32/64-bit compilers (e.g. GCC/Turbo C).

Chapter Summary

- Qualifiers modify size/range/sign of basic types: `short`, `long`, `signed`, `unsigned`.
- `signed` is the default; `unsigned` drops negative range to double the positive range.
- `short` reduces size, `long` increases size and range.

6 Escape Sequences

Definition: An escape sequence is a combination of a backslash `\` followed by a character, used to represent special, non-printable, or otherwise hard-to-type characters within character or string constants.

6.1 Escape Sequence Reference Table

Sequence	Meaning	Purpose / Usage
`\n`	New line	Moves the cursor to the beginning of the next line
`\t`	Horizontal tab	Inserts a horizontal tab space (for alignment)
`\b`	Backspace	Moves the cursor one position back
`\r`	Carriage return	Moves the cursor to the beginning of the current line
`\\`	Backslash	Prints a literal backslash character
`\"`	Double quote	Prints a double quote inside a string literal
`\'`	Single quote	Prints a single quote inside a character literal
`\a`	Alert (bell)	Produces an audible beep sound
`\f`	Form feed	Moves to the start of the next logical page (printer)
`\v`	Vertical tab	Inserts a vertical tab space
`\0`	Null character	Marks the end of a string (used internally)

6.2 Examples with Output

EXAMPLE

```
#include <stdio.h>
int main() {
    printf("Name:\tRahul\n");
    printf("Path: C:\\Users\\Rahul\n");
    printf("She said, \"Good Morning\"\n");
    return 0;
}
```

OUTPUT

```
Name:   Rahul
Path: C:\Users\Rahul
She said, "Good Morning"
```

Common Mistake

Writing a raw backslash for a Windows-style path such as `"C:\Users"` — `\U` is not a recognized sequence, so each backslash must be escaped as `\\`.

Chapter Summary

- Escape sequences = `\` + character, used for special/non-printable characters inside constants.
- Most common: `\n` (new line), `\t` (tab), `\b` (backspace), `\\`, `\"`, `\'`.
- `\0` (null character) is used internally to terminate strings.

7 Arithmetic Expressions and Operators

7.1 Operator Classification Chart

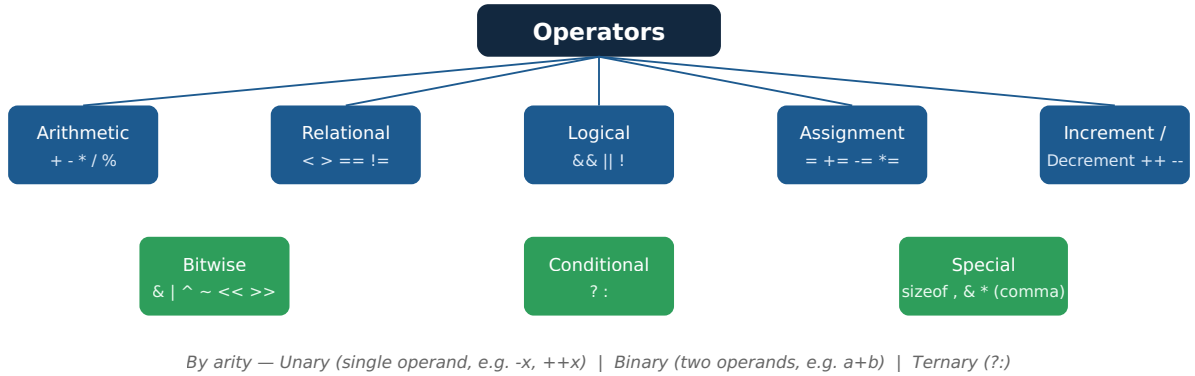


Fig. 7.1 — Classification of C Operators

7.2 Arithmetic Operators

Operator	Meaning	Example (a=10,b=3)	Result
`+`	Addition	`a+b`	13
`-`	Subtraction	`a-b`	7
`*`	Multiplication	`a*b`	30
`/`	Division (integer division truncates)	`a/b`	3
`%`	Modulus (remainder) — integers only	`a%b`	1

7.3 Unary and Binary Operators

- **Unary operators** act on a single operand: unary minus `-x`, increment `++x` / `x++`, decrement `--x` / `x--`, logical NOT `!x`, address-of `&x`, `sizeof(x)`.
- **Binary operators** act on two operands: all arithmetic (`+ - * / %`), relational, logical (`&& ||`), assignment, and bitwise operators.
- **Ternary operator** — the only operator taking 3 operands: `condition ? expr1 : expr2`.

7.4 Operator Precedence and Associativity

Precedence	Operator(s)	Associativity
Highest	`()` `[]` `.` `->`	Left to Right
2	`!` `~` `++` `--` unary `+` `-` `*`(deref) `&` `sizeof`	Right to Left
3	`*` `/` `%`	Left to Right
4	`+` `-` (binary)	Left to Right
5	`<<` `>>`	Left to Right
6	`<` `<=` `>` `>=`	Left to Right
7	`==` `!=`	Left to Right
8	`&` (bitwise AND)	Left to Right
9	`^`	Left to Right
10	`|`	Left to Right
11	`&&`	Left to Right
12	`||`	Left to Right
13	`?:` (ternary)	Right to Left
14	`=` `+=` `-=` etc.	Right to Left
Lowest	`,` (comma)	Left to Right

Exam Tip

Multiplication/Division/Modulus (`* / %`) always bind tighter than Addition/Subtraction. When operators share precedence, evaluation follows associativity (mostly left-to-right, but assignment and unary operators are right-to-left).

7.5 Expression Evaluation — Sample Program

EXAMPLE

```
#include <stdio.h>
int main() {
    int a = 8, b = 3, c;
    c = a + b * 2 - a / b;
    printf("Result = %d\n", c);   // 8 + (3*2) - (8/3) = 8+6-2 = 12
    return 0;
}
```

OUTPUT

```
Result = 12
```

Chapter Summary

- Operator categories: Arithmetic, Relational, Logical, Assignment, Increment/Decrement, Bitwise, Conditional, Special (sizeof, comma, address-of).
- By arity: Unary (1 operand), Binary (2 operands), Ternary (3 operands — only `?:`).
- `* / %` have higher precedence than `+ -`; assignment/unary operators associate right-to-left.

8 Preprocessor Directives

8.1 Introduction to the Preprocessor

The **preprocessor** is a program that processes the source code **before** it is handed to the compiler. Preprocessor directives always begin with a `#` symbol, do not end with a semicolon, and are handled by the preprocessor, not the compiler itself.



Fig. 8.1 — How the Preprocessor Works

8.2 #include Directive

Instructs the preprocessor to insert the entire contents of a specified header file at that point in the program.

SYNTAX

```
#include <filename> // for standard library headers #include "filename" // for user-defined / local headers
```

EXAMPLE

```
#include <stdio.h>
#include "myheader.h"
```

8.3 #define Directive

Used to define a **macro** — either a symbolic constant or a macro with arguments — that the preprocessor textually substitutes wherever it appears in the code.

SYNTAX

```
#define IDENTIFIER value #define MACRO_NAME(args) expression
```

EXAMPLE

```
#define PI 3.14159
#define SQUARE(x) ((x)*(x))

int main() {
    printf("%f", PI);
    printf("%d", SQUARE(5));   // expands to ((5)*(5))
    return 0;
}
```

8.4 Conditional Compilation

Allows parts of code to be compiled selectively based on a condition, using directives such as `#ifdef`, `#ifndef`, `#if`, `#else`, and `#endif`. This is commonly used to prevent multiple inclusion of the same header file (include guards) or to write platform-specific code.

EXAMPLE — INCLUDE GUARD

```
#ifndef MYHEADER_H
#define MYHEADER_H
   // header content
#endif
```

8.5 Advantages of Preprocessor Directives

- Improves **code readability** using symbolic names instead of raw values.
- Enables **code reuse** by including shared header files.
- Supports **conditional/portable compilation** across platforms.
- Macros can act like inline functions, potentially improving performance (no function-call overhead).

Chapter Summary

- Preprocessor directives start with `#` and are processed before compilation.
- `#include` inserts header file content; `#define` creates macros/symbolic constants.
- Conditional compilation (`#ifdef/#ifndef/#endif`) selectively includes code.

9 Header Files

9.1 Concept of Header Files

A **header file** is a file (usually with extension `.h`) that contains function prototypes (declarations), macro definitions, and type definitions used by the standard library or user programs. Header files are pulled into the source file by the `#include` directive so that the compiler knows the signatures of the functions being used.

9.2 Standard Header Files — Common Examples

Header File	Purpose / Contains
`stdio.h`	Standard input/output functions — `printf`, `scanf`
`stdlib.h`	General utilities — `malloc`, `exit`, `rand`
`string.h`	String handling — `strlen`, `strcpy`, `strcmp`
`math.h`	Mathematical functions — `sqrt`, `pow`, `sin`
`ctype.h`	Character testing/conversion — `isalpha`, `toupper`
`time.h`	Date and time functions
`conio.h`	Console I/O (compiler-specific, e.g. Turbo C) — `clrscr`, `getch`

9.3 Purpose and Usage

- Provides **function prototypes**, letting the compiler check calls for correct argument types/count.
- Promotes **modularity** — programs can be split across files sharing common declarations.
- Distinguishes **system headers** (`<...>`, searched in standard directories) from **user headers** (`"..."`, searched in the current directory first).

> **Note**
> Header files themselves do not contain the actual function code (definitions) for library functions — only declarations. The compiled function code resides in pre-built library files linked in at the linking stage.

Chapter Summary

- Header files (.h) hold declarations/macros needed by the compiler; included via `#include`.
- `<header.h>` = standard library; `"header.h"` = user-defined/local file.
- Common headers: stdio.h, stdlib.h, string.h, math.h, ctype.h, time.h.

10 Symbolic Constants

10.1 Definition

A **symbolic constant** is a name that represents a fixed value that never changes during program execution. It is defined using the `#define` preprocessor directive (or the `const` keyword) and, by convention, is written in **UPPERCASE** to distinguish it from ordinary variables.

10.2 Creating Symbolic Constants using #define

SYNTAX

```
#define SYMBOLIC_NAME value
```

EXAMPLE

```
#include <stdio.h>
#define PI 3.14159
#define MAX_MARKS 100

int main() {
    float area, radius = 2.0;
    area = PI * radius * radius;
    printf("Area = %.2f\n", area);
    printf("Maximum Marks = %d\n", MAX_MARKS);
    return 0;
}
```

OUTPUT

```
Area = 12.57
Maximum Marks = 100
```

10.3 Benefits and Applications

- **Readability** — `MAX_MARKS` is more meaningful than the literal `100`.
- **Easy maintenance** — changing the value in one `#define` line updates it everywhere it is used.
- **Avoids "magic numbers"** scattered throughout code, reducing errors.
- Substitution happens at **compile time** (via the preprocessor), so there is no runtime memory overhead like a variable.

Exam Tip

A symbolic constant defined with `#define` is *not* a variable — it has no data type and no memory address; it is a pure text substitution performed before compilation.

Chapter Summary

- Symbolic constants = named, fixed values created via `#define` (or `const`), written in UPPERCASE by convention.
- Improve readability, ease maintenance, and avoid magic numbers.

11 Comments in C

11.1 Purpose of Comments

Comments are non-executable statements that explain the code to human readers. They are completely ignored by the compiler and do not affect the program's output or performance.

11.2 Single-line Comments

Begin with `//` and extend to the end of that line. (Supported from C99 onward; widely supported by virtually all modern compilers.)

EXAMPLE

```
// This program calculates the area of a circle
int radius = 5;   // radius in centimetres
```

11.3 Multi-line Comments

Begin with `/*` and end with `*/`; can span multiple lines. This style is valid in **all** versions of C, including strict ANSI C89.

EXAMPLE

```
/* This is a multi-line comment
   describing the purpose of the
   function below */
int add(int a, int b) {
    return a + b;
}
```

11.4 Importance and Best Practices

- Improves code **readability** and helps other programmers (or your future self) understand the logic.
- Useful for **documentation** — author name, date, purpose of a function/module.
- Can be used to **temporarily disable** code during debugging/testing.

Common Mistake

Multi-line comments **cannot be nested** — writing `/* outer /* inner */ still-outside */` causes a compile error because the first `*/` closes the comment early.

Basis	Single-line `//`	Multi-line `/* */`
Span	One line only	Multiple lines
ANSI C89 support	Not part of strict ANSI C89 (compiler extension)	Fully supported
Nesting	N/A (each line independent)	Cannot be nested

Chapter Summary

- Comments are ignored by the compiler; used purely for human-readable documentation.
- `//` for single-line; `/* ... */` for multi-line (works everywhere, cannot nest).

12 The sizeof Operator

12.1 Definition and Syntax

`sizeof` is a **compile-time unary operator** that returns the amount of memory (in bytes) occupied by a variable, data type, or expression. It returns a value of type `size_t` (an unsigned integer type).

SYNTAX

```
sizeof(data_type) sizeof(variable_name) sizeof variable_name // parentheses optional for variables
```

12.2 Determining Memory Requirements — Example

EXAMPLE

```
#include <stdio.h>
int main() {
    int a;
    float b;
    double c;
    char d;
    printf("int: %lu bytes\n", sizeof(a));
    printf("float: %lu bytes\n", sizeof(b));
    printf("double: %lu bytes\n", sizeof(c));
    printf("char: %lu bytes\n", sizeof(d));
    return 0;
}
```

OUTPUT (TYPICAL, 32/64-BIT SYSTEM)

```
int: 4 bytes
float: 4 bytes
double: 8 bytes
char: 1 bytes
```

12.3 Practical Applications

- Writing **portable code** — instead of assuming a fixed size, use `sizeof` so the program adapts to the compiler/platform.
- Calculating the number of elements in an array: `sizeof(arr)/sizeof(arr[0])`.
- Determining memory to allocate dynamically, e.g. `malloc(n * sizeof(int))`.
- Comparing storage requirements of different data types/qualifiers.

Exam Tip

`sizeof` is evaluated at **compile time** (except for variable-length arrays in C99), so it does *not* evaluate side effects inside its operand — e.g. in `sizeof(x++)`, `x` is *not* actually incremented.

Chapter Summary

- `sizeof` = compile-time unary operator returning memory size in bytes.
- Works on data types, variables, and expressions; result type is `size_t`.
- Key uses: portability, array-length calculation, dynamic memory allocation.

13 Translation Process of a C Program

Converting human-readable C source code into an executable program that the machine can run involves several distinct stages, collectively called the **translation process**.

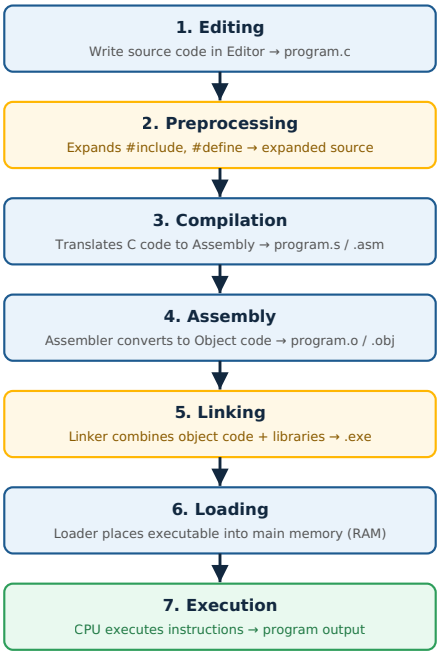


Fig. 13.1 — Translation Stages of a C Program

13.1 Stage-wise Explanation

Stage	Performed By	Input	Output
1. Editing	Programmer / Text Editor / IDE	Idea / logic	Source file (`.c`)
2. Preprocessing	Preprocessor	Source file	Expanded source code
3. Compilation	Compiler	Expanded source code	Assembly code (`.s`)
4. Assembly	Assembler	Assembly code	Object code (`.obj` / `.o`)
5. Linking	Linker	Object code + library files	Executable file (`.exe`)
6. Loading	Loader (part of OS)	Executable file	Program loaded into RAM
7. Execution	CPU	Loaded program in memory	Program output

Details of Each Stage

- **Editing:** The programmer types the source code using a text editor or IDE and saves it with a `.c` extension.
- **Preprocessing:** The preprocessor scans for directives (`#include`, `#define`), expanding/replacing them, and strips out comments.

- **Compilation:** The compiler checks syntax and semantics, then translates the expanded C code into low-level assembly language specific to the target processor.
- **Assembly:** The assembler converts the human-readable assembly instructions into machine-readable object code (binary, but not yet executable — external references are unresolved).
- **Linking:** The linker resolves references to library functions (e.g. `printf` from the C standard library) and combines multiple object files into a single executable.
- **Loading:** The operating system's loader copies the executable into main memory (RAM) and sets up the runtime environment before execution.
- **Execution:** The CPU fetches, decodes, and executes the loaded machine instructions, producing the program's output.

> **Note**
>
> Some textbooks combine Assembly and Compilation into a single "Compilation" stage that directly produces object code — both views are acceptable; know the 7-stage breakdown for detailed questions and the shorter 4-stage version (Preprocessing → Compilation → Linking → Loading/Execution) for quick-answer questions.

Chapter Summary

- Translation order: Editing → Preprocessing → Compilation → Assembly → Linking → Loading → Execution.
- Preprocessor expands directives; Compiler produces assembly; Assembler produces object code; Linker resolves libraries and produces the executable; Loader brings it into RAM; CPU executes it.

14 Quick Revision Section

14.1 Important Definitions (One-Line Recall)

Term	One-line Definition
Character Set	Set of valid characters (alphabets, digits, symbols, white spaces) a compiler recognizes
Identifier	User-defined name for a variable, function, or other program element
Keyword	Reserved word with fixed meaning; cannot be used as an identifier
Data Type	Specifies type, size, and range of values a variable can hold
Constant	A value that does not change during program execution
Type Qualifier	Keyword (short/long/signed/unsigned) that modifies size/range/sign of a type
Escape Sequence	Backslash + character combination representing a special/non-printable character
Operator	Symbol that performs an operation on one or more operands
Preprocessor Directive	Instruction (starting with #) processed before compilation
Header File	File containing declarations/macros included via #include
Symbolic Constant	Named fixed value created using #define, in UPPERCASE by convention
Comment	Non-executable text ignored by the compiler, for documentation
`sizeof`	Compile-time unary operator returning memory size (bytes) of its operand

14.2 Keyword List (All 32 — ANSI C)

```
auto   break   case   char   const   continue   default   do   double   else
enum   extern   float   for   goto   if   int   long   register   return
short   signed   sizeof   static   struct   switch   typedef   union
unsigned   void   volatile   while
```

14.3 Escape Sequence Table (Quick Recall)

\n	\t	\b	\r	\\	\"	\'	\a	\0
New line	Tab	Backspace	Carriage return	Backslash	Double quote	Single quote	Alert/ beep	Null char

14.4 Data Type Summary Table

Type	Size	Specifier	Category
char	1 byte	%c	Primary
int	4 bytes	%d	Primary
float	4 bytes	%f	Primary
double	8 bytes	%lf	Primary
void	—	—	Primary (no value)
array, pointer, function	varies	—	Derived
struct, union, enum, typedef	varies	—	User-defined

14.5 Operator Precedence — Quick Table (High → Low)

1	`() [] . ->`
2	`! ~ ++ -- unary+/- * & sizeof`
3	`* / %`
4	`+ -`
5	`<< >>`
6	`< <= > >=`
7	`== !=`
8-10	`& ^ |`
11-12	`&& ||`
13	`?:`
14	`= += -= ...`
15	`,`

14.6 Frequently Asked Exam Questions

Q1. Who developed C, and when?
A. Dennis Ritchie, at Bell Labs, in 1972.

Q2. How many keywords does ANSI C define?
A. 32 keywords.

Q3. What is the difference between `signed` and `unsigned`?
A. `signed` stores both positive and negative values; `unsigned` stores only non-negative values, doubling the positive range.

Q4. What is the use of the `\n` escape sequence?
A. It moves the cursor to the beginning of the next line (newline).

Q5. What is the difference between `#include <stdio.h>` and `#include "myfile.h"`?
A. Angle brackets search standard system directories for library headers; quotes search the current/local directory first for user-defined headers.

Q6. What does the `sizeof` operator return?
A. The number of bytes occupied by a variable, type, or expression, evaluated at compile time.

Q7. List the stages in translating a C program into an executable.
A. Editing → Preprocessing → Compilation → Assembly → Linking → Loading → Execution.

Q8. Why are symbolic constants written in uppercase?
A. By convention, to visually distinguish them from ordinary variables (not a compiler rule, but a widely followed best practice).

Q9. Can a keyword be used as a variable name?
A. No — keywords are reserved and produce a compilation error if used as identifiers.

Q10. Which operator has the highest precedence among arithmetic operators?
A. `* / %` (equal precedence, higher than binary `+ -`), evaluated left to right.

14.7 One-Page Revision Notes

C Fundamentals — At a Glance

- **Character Set:** Alphabets + Digits + Special Symbols + White spaces.
- **Identifiers:** Start with letter/underscore, only letters/digits/underscore, case-sensitive, 31 significant characters (ANSI C).
- **Keywords:** 32 reserved lowercase words in ANSI C — cannot be redefined.
- **Data Types:** Primary (int, float, double, char, void); Derived (array, pointer, function); User-defined (struct, union, enum, typedef).
- **Constants:** Integer, Floating-point, Character, String.
- **Qualifiers:** short, long, signed (default), unsigned — modify size/range/sign.
- **Escape Sequences:** \n \t \b \r \\ \" \' \a \0 — represent special/non-printable characters.
- **Operators:** Arithmetic, Relational, Logical, Assignment, Increment/Decrement, Bitwise, Conditional, Special; classified as Unary/Binary/Ternary; follow strict precedence and associativity rules.
- **Preprocessor:** #include (inserts headers), #define (creates macros/constants), conditional compilation (#ifdef/#ifndef/#endif) — processed before compilation.
- **Header Files:** .h files with declarations; <> for standard, "" for user-defined.
- **Symbolic Constants:** Named fixed values via #define, UPPERCASE convention, improve readability & maintainability.
- **Comments:** // single-line; /* */ multi-line (cannot nest); ignored by compiler.
- **sizeof:** Compile-time operator; returns size in bytes; used for portability, arrays, dynamic memory.
- **Translation Process:** Editing → Preprocessing → Compilation → Assembly → Linking → Loading → Execution.